Day 58: Read Account Summary from File

Day 58: Read Account Summary from File

1. Day Number

Day 58

2. Topic Name

File Reading with Account Summary

Today you will learn how to **open a text file, read its contents, and display the information line by line**.

---

3. Connection with Yesterday

Yesterday, on **Day 57**, you saved an account summary into a text file.

For example, the file might contain:

```
Account Holder: Radhe
Balance: 5000
```

Today you will do the opposite:

```
File
 ↓
Open file
 ↓
Read each line
 ↓
Clean the line
 ↓
Print it
```

So the flow is:

```
Day 57
Python data → Text file

Day 58
Text file → Python → Screen
```

---

4. Important Topics

Today's main topics are:

- file reading
- `with`
- looping through a file
- `strip()`

The most important pattern is conceptually:

```
with open(...) as file:
    for line in file:
        ...
```

You don't need to memorize it immediately. Understand what each part does.

---

5. Foundational Notes

A. What is file reading?

File reading means getting previously stored information from a file.

Suppose a file called:

```
account_summary.txt
```

contains:

```
Account Holder: Radhe
Balance: 5000
```

Python can open this file and read those lines.

B. Read mode "r"

When opening a file for reading, we normally use:

```
"r"
```

Here:

```
r = read
```

Conceptually:

```
open("account_summary.txt", "r")
```

means:

> Open `account_summary.txt` because I want to read its contents.

C. Why use with?

You will commonly see:

```
with open(...) as file:
```

`with` is useful because Python automatically closes the file when you finish using it.

Without `with`, you normally have to remember to close the file yourself.

Think of it like:

```
with
 ↓
Open file
 ↓
Use file
 ↓
Automatically close file
```

For beginner Python, prefer the `with` approach.

D. A file can be looped over

A text file contains lines.

For example:

```
Account Holder: Radhe
Balance: 5000
```

Python lets you process these lines one at a time:

```
First loop:
Account Holder: Radhe

Second loop:
Balance: 5000
```

This is useful when a file contains many lines.

E. What does strip() do?

Lines read from a text file often contain an invisible newline character:

```
"\n"
```

For example, Python may internally read:

```
"Account Holder: Radhe\n"
```

Using:

```
line.strip()
```

removes extra whitespace around the line, including the newline.

So:

```
"Account Holder: Radhe\n"
```

becomes:

```
"Account Holder: Radhe"
```

F. Why can printing without strip() look strange?

Suppose a line already ends with:

```
\n
```

and then `print()` also adds a newline.

You could get output that looks like:

```
Account Holder: Radhe

Balance: 5000
```

Notice the unwanted empty line.

Using `strip()` helps produce:

```
Account Holder: Radhe
Balance: 5000
```

---

6. Easy Example

Suppose a file named:

```
colors.txt
```

contains:

```
Red
Green
Blue
```

The general idea is:

```
with open("colors.txt", "r") as file:
    for color in file:
        print(color.strip())
```

Output:

```
Red
Green
Blue
```

Notice the steps:

```
Open file
   ↓
```

```
Loop through file
   ↓
Get one line
   ↓
Remove newline using strip()
   ↓
Print line
```

This example demonstrates the technique you need today without solving the account-summary problem for you.

---

7. Problem Statement

Create a Python program that reads an existing account summary from a text file.

Assume the file contains information such as:

```
Account Holder: Radhe
Balance: 5000
```

Your program should:

1. Open the account summary file in **read mode**.
2. Read the file **line by line**.
3. Use `strip()` to remove unnecessary whitespace/newlines.
4. Print each line.

Do not rewrite the file.

Your task today is only:

```
File → Read → Display
```

---

8. Concepts Used

You will use:

File handling

To access information stored in a file.

open()

Used to open a file.

"r"

Specifies that the file is being opened for reading.

with

Automatically manages opening and closing the file.

for loop

Processes one file line at a time.

strip()

Removes unwanted spaces and newline characters from the beginning and end of a string.

print()

Displays each cleaned line.

---

9. Thought Process

Before writing code, think through the problem.

Step 1: What information do I need?

The account summary already exists inside a text file.

You don't need to create the account data again.

Step 2: Am I writing or reading?

Today you are reading.

Therefore think:

```
mode = "r"
```

not:

```
mode = "w"
```

Be careful: `"w"` can overwrite a file.

Step 3: How should I open the file?

Use the safer beginner-friendly approach:

```
with open(...)
```

Step 4: How should I process the contents?

The requirement says:

> print it line by line

Therefore a loop is a natural choice.

Conceptually:

```
for every line in file
```

Step 5: What should I do with each line?

Each line may contain `\n`.

Therefore:

```
clean_line = line.strip()
```

Then display it.

Complete mental flow

```
Start
  ↓
Choose filename
  ↓
Open file in read mode
  ↓
Get first line
  ↓
Remove newline/extra whitespace
  ↓
Print line
  ↓
More lines?
 ┌─────────┴─────────┐
Yes                 No
 ↓                   ↓
Repeat               End
```

---

10. Pseudocode

```
START

SET filename to account summary file

OPEN filename in read mode using with

    FOR each line in file

        REMOVE extra whitespace from line

        PRINT cleaned line

END FOR

END
```

Notice that pseudocode describes **what to do**, not exact Python syntax.

---

11. Suggested Solving Approach

For today, I recommend the **functional/simple procedural approach**.

You don't really need a class just to read and display a small file.

A simple structure is enough:

```
Open file
→ loop
→ strip
→ print
```

Later, if your banking project becomes larger, you could put file behavior inside methods such as:

```
BankAccount
 ├── deposit()
 ├── withdraw()
 ├── save_summary()
 └── load_summary()
```

But don't make today's exercise unnecessarily complicated.

---

12. Easy Edge Cases

Edge Case 1: Empty File

Suppose:

```
account_summary.txt
```

exists but contains nothing.

Then the loop has no lines to process.

So the program may simply print nothing.

Conceptually:

```
File exists
 ↓
0 lines
 ↓
Loop runs 0 times
```

This is not necessarily an error.

Edge Case 2: File Not Found

Suppose Python tries to open:

```
account_summary.txt
```

but that file does not exist.

Python can raise:

```
FileNotFoundError
```

For today's beginner exercise, you only need to understand the idea.

Later you can handle it using:

```
try:
    ...
except FileNotFoundError:
    ...
```

You do **not** need to make exception handling the main focus today.

Edge Case 3: Blank Line Inside File

Suppose the file contains:

```
Account Holder: Radhe

Balance: 5000
```

One of the lines is empty.

After:

```
strip()
```

that line becomes an empty string:

```
""
```

For today's exercise, printing it is acceptable.

---

13. Expected Input

The input is mainly the existing text file.

Example file:

`account_summary.txt`

```
Account Holder: Radhe
Balance: 5000
```

You do not necessarily need `input()` from the keyboard.

The file itself is the source of data.

---

14. Expected Output

For the example above:

```
Account Holder: Radhe
Balance: 5000
```

Another example:

Input file:

```
Account Holder: Aman
Balance: 0
```

Expected screen output:

```
Account Holder: Aman
Balance: 0
```

---

15. Hint Only

Focus on these four pieces:

```
with open(..., "r") as file:
```

then think:

```
for ... in file:
```

then:

```
...strip()
```

and finally:

```
print(...)
```

Try to combine those ideas yourself.

Your target logic

```
account_summary.txt
        ↓
    open with "r"
        ↓
   loop over lines
        ↓
      strip()
        ↓
      print()
```

Do not use `"w"` today, because your goal is to read the account summary, not overwrite it.